

M&A Multi-Agent Decision Support Backend

Live Demo: <https://merger-and-acquisition.streamlit.app/>

M&A Multi-Agent Decision Support Backend

Project Report - Academic Submission

Programme: Analyst-Assisted Decision Support Prototype

Case Default: Tata Steel (Acquirer) -> Jindal Steel (Target, JINDALSTEL.NS)

Date: 10 October 2026

Repository Folder: outputs/Merger-and-acquisition

Stack: Python 3.10+, Streamlit, yfinance, pandas, Groq LLM API, Docker

Live Demo: <https://merger-and-acquisition.streamlit.app/>

Report Type: Comprehensive, PDF-print-ready (Markdown source)

Status disclaimer: Academic decision-support prototype. Does not verify source documents, produce an investment recommendation, approve transactions, or replace legal, accounting, tax, commercial, environmental, or investment-banking diligence. All Yahoo Finance values and model-generated text require human review and primary-source verification.

Table of Contents

1. Abstract
 2. 1. Introduction and Background
 3. 2. Objectives
 4. 3. System Architecture
 5. 4. Methodology and Workflow
 6. 5. Module-by-Module Breakdown
 7. 6. Data Sources and Valuation Approach
 8. 7. Setup, Execution and Deployment
 9. 8. Notebook Review Findings Addressed
 10. 9. Risks, Ethics, Limitations and Controls
 11. 10. Future Work and Roadmap
 12. Appendix A: File Inventory
 13. Appendix B: Output Schema
 14. Appendix C: References
-

Abstract

This project reorganises an exploratory M&A notebook into a small, maintainable Python backend with two interfaces: a Streamlit dashboard (app.py) and a command-line runner (run.py). The core logic lives in the `ma_agents` package, which implements a six-stage multi-agent pipeline: (1) Financial Due Diligence, (2) Valuation, (3) Industry Intelligence, (4) Synergy Analysis, (5) Red-Team Risk Review, and (6) Investment Committee Discussion Memo.

Deterministic data tools in `ma_agents/tools.py` fetch latest available financial statements via `yfinance` and compute ratios and a simplified EBITDA-based valuation illustration. Specialist report writers in `ma_agents/agents.py` call the Groq Chat Completions API with `temperature=0` and a bounded response budget. The orchestrator in `ma_agents/orchestrator.py` runs each specialist once and forwards already-generated reports to the review stages, avoiding the notebook's repeated fetches and model calls. Configuration in `ma_agents/config.py` loads `GROQ_API_KEY`, `GROQ_MODEL`, and `MAX_COMPLETION_TOKENS` from environment or Streamlit Secrets. Deployment is supported locally, on Streamlit Community Cloud, and via Docker.

Industry and synergy content remain qualitative and hypothetical by design, and the valuation is explicitly labelled as an academic illustration, not a professional FCFF DCF.

1. Introduction and Background

Mergers and Acquisitions analysis typically spans financial diligence, valuation, industry context, synergy estimation, independent risk challenge, and committee deliberation. The source notebook demonstrated these roles interactively but had structural weaknesses for reuse: inline pip install, an embedded Groq API key, a naming mismatch (`financial_dd_tool` vs `financial_due_diligence`), repeated data fetches and model calls in the red-team stage, and a

simplified DCF that discounted EBITDA without clearly stating its limits (see README.md:79-86).

This backend preserves the six roles while:

- Separating deterministic computation (ratios, multiples) from LLM interpretation.
- Running each stage once per transaction.
- Handling missing data, zero denominators, empty statements, statement period, and token limits.
- Externalising secrets and runtime dependencies.
- Providing reproducible CLI, dashboard, and container entry points.

Default demonstration transaction throughout code and docs is Tata Steel acquiring Jindal Steel (JINDALSTEL.NS).

2. Objectives

Primary objectives:

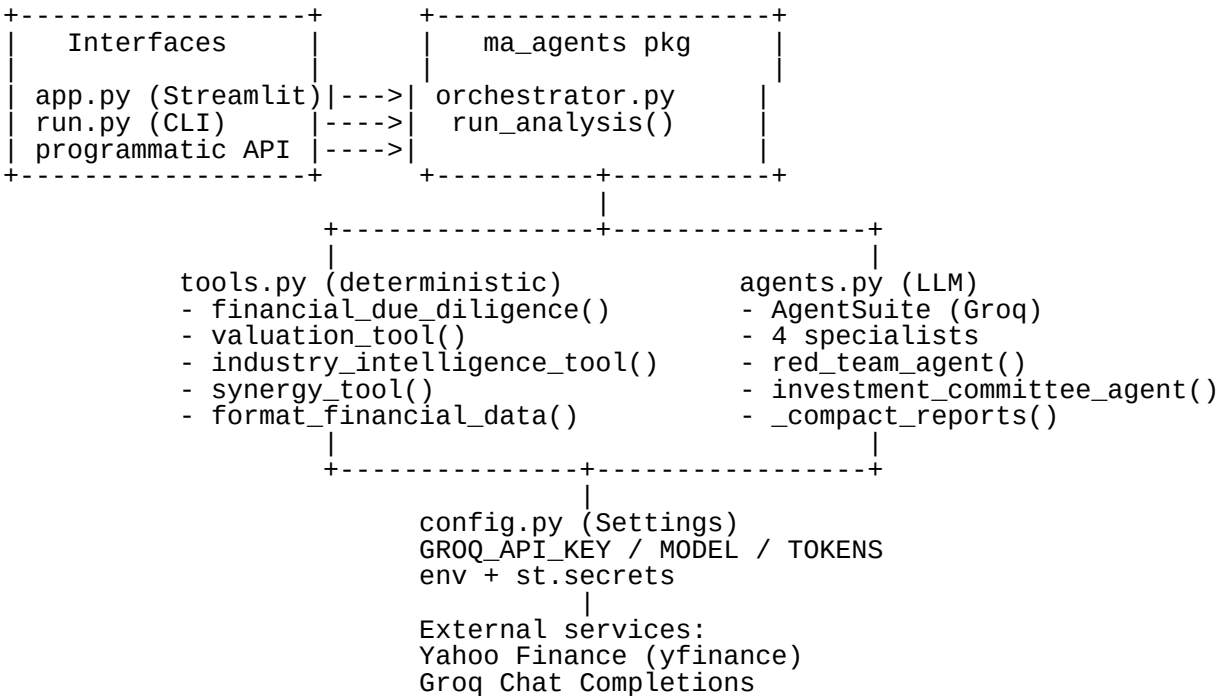
1. Preserve notebook specialist roles: financial diligence, valuation, industry, synergy, red-team, investment committee.
2. Fetch latest available statements and metadata from Yahoo Finance via yfinance.
3. Calculate basic ratios and a transparent academic valuation illustration.
4. Generate six structured reports via Groq with traceable, bounded prompts.
5. Save / export reports to JSON for downstream review.
6. Support local dashboard, CLI, programmatic import, and container deployment.

Non-objectives (explicitly out of scope per README.md:88-90):

- No investment recommendation or approval.
- No verification of source documents.
- No replacement for professional diligence functions.
- No access control in the dashboard - authentication must be added before public exposure of sensitive deals.

3. System Architecture

3.1 High-level components



3.2 Data flow per transaction

1. Input: target_ticker, acquirer, target strings.
2. get_settings() validates GROQ_API_KEY, resolves model and token budget.
3. Parallel-conceptual (sequential in code) tool fetches:

- 4. financial_due_diligence(ticker) -> dict of amounts, margins, leverage, flags.
- 5. valuation_tool(ticker) -> formatted string with EV, multiples, DCF-style value.
- 6. industry_intelligence_tool(acquirer, target) -> qualitative Indian steel context template.
- 7. synergy_tool(acquirer, target) -> hypothetical synergy categories template.
- 8. AgentSuite produces four specialist markdown reports.
- 9. _compact_reports() truncates each to 2400 chars for red-team / committee context to stay below provider TPM limits.
- 10. Red-team and committee agents produce final two reports.
- 11. Caller persists dict of six reports to JSON (reports/*.json or Streamlit download).

3.3 Design principles

- **Single execution:** Each specialist runs once; reports are passed forward, not regenerated.
- **Determinism first:** Numbers come from yfinance + pandas logic; LLM only interprets supplied text and must not invent figures.
- **Fail-safe defaults:** math.nan -> N/A, zero-division guarded, empty income statement raises clear ValueError.
- **Cost control:** Default 1000 tokens locally, 700 on Cloud; chained-report shortening reduces request size.

4. Methodology and Workflow

4.1 Six-stage pipeline (ma_agents/orchestrator.py:14-32)

```
reports = {
    "Financial Due Diligence": agents.financial_agent(...),
    "Valuation": agents.valuation_agent(...),
    "Industry Intelligence": agents.industry_agent(...),
    "Synergy Analysis": agents.synergy_agent(...),
}
reports["Red-Team Risk Review"] = agents.red_team_agent(...)
reports["Investment Committee Discussion Memo"] = agents.investment_committee_agent(...)
```

Each call uses temperature=0 for reproducibility (ma_agents/agents.py:25-32).

4.2 Prompt methodology

Agent	System role			User prompt constraints	Output sections
Financial	Senior financial DD analyst			Use only supplied info, separate facts vs interpretation, flag missing data, no recommendation	Executive summary, performance, profitability, debt/leverage, cash flow/liquidity, strengths, DD questions, assessment
Valuation	Senior analyst	M&A	valuation	Explain multiples vs DCF-illustration, assumptions + sensitivity, info required for professional valuation	Summary, market valuation, EV/revenue, EV/EBITDA, DCF-style, assumptions, risks, info required, conclusion
Industry	Industry & intelligence	competitive		Separate general factors from verified facts, no invented shares, identify sourcing gaps	Overview, demand, cyclicity, raw materials/energy, competition, risks, regulatory/environmental, strategic considerations, gaps
Synergy	Synergy / PMO analyst			Hypothetical only, no invented numbers, evidence + integration risks	Summary, cost/revenue ops, operations, realisation requirements, risks, info required, assessment

Agent	System role	User prompt constraints	Output sections
Red-team	Independent risk / challenger	Find unsupported conclusions, inconsistencies, downside scenarios	Risk summary, financial/valuation/industry/synergy risks, weak assumptions, gaps, worst-case, management questions
Committee	Memo editor	Based only on reports below, no approval decision, stress verification	Transaction summary, rationale, diligence, valuation, synergies, context, risks, gaps, scenarios, questions, discussion points

4.3 Context bounding

`_compact_reports(reports, 2400)` (`ma_agents/agents.py:8-16`) truncates at word boundary and appends [Report excerpt shortened...] notice. This directly addresses Groq Request too large / rate_limit_exceeded / tokens per minute errors surfaced in `app.py:81-83`.

5. Module-by-Module Breakdown

5.1 app.py - Streamlit dashboard (125 lines)

- Loads `.env`, sets page config M&A Deal Desk wide layout.
- Custom CSS hero (Analyst workspace), 3 metrics (Analysis sections, Specialist reports=4, Review stages=2).
- Sidebar form transaction_form: Acquirer, Target, Ticker + Generate analysis primary button.
- On submit: validates non-empty, shows `st.status` pipeline, calls `run_analysis()`, stores in `st.session_state["deal_analysis"]` with UTC timestamp.
- Error handling: missing `GROQ_API_KEY` -> Secrets fix instructions; TPM / too-large -> lower `MAX_COMPLETION_TOKENS`; generic -> check `key/deps/network/ticker`.
- Display: `st.tabs` per report name, markdown render, JSON download `ticker-ma-analysis.json`.
- Empty state explains 6 deliverables; footer disclaimer: academic prototype, not recommendation.

5.2 run.py - CLI (26 lines)

- `argparse` defaults: `--ticker JINDALSTEL.NS --acquirer Tata Steel --target Jindal Steel --output reports/analysis.json`.
- `load_dotenv()`, `run_analysis()`, `json.dump(..., indent=2, ensure_ascii=False)`, prints Saved N reports to
- Suitable for batch / CI / headless runs requiring network + credentials.

5.3 ma_agents/__init__.py (5 lines)

- Public API: `from .orchestrator import run_analysis; __all__=["run_analysis"]`.
- Backend use: `from ma_agents import run_analysis`.

5.4 ma_agents/config.py (56 lines)

- Frozen dataclass `Settings(groq_api_key, model="openai/gpt-oss-20b", max_completion_tokens=1000)`.
- `_clean()` strips whitespace and surrounding single/double quotes (TOML copy-paste tolerance).
- `get_settings()`: precedence `env` -> `st.secrets` -> default; raises `RuntimeError` with local + Cloud fix if key missing.
- `MAX_COMPLETION_TOKENS` parsed as int.

5.5 ma_agents/tools.py (144 lines) - Deterministic layer

- Helpers: `_number()`, `_row_value(frame,row)`, `_fmt(value,currency="Rs.")`, `local_ratio()`.
- `financial_due_diligence(ticker)`:
- `yf.Ticker(ticker).info` / `income_stmt` / `balance_sheet` / `cashflow`.
- Extracts Revenue, Operating Income, Net Income, EBITDA (fallback Normalized EBITDA), Debt, Cash, Equity, Assets, Liabilities, OCF, Capex.

- Computes Operating/EBITDA/Net margins, Debt/Equity, Debt/Operating Income, OCF/Debt, Net Debt.
- Flags: High D/E >1, Low EBITDA margin <10%, Net Debt >0, Negative OCF; else No major automated flags (limited data caveat applies).
- `financial_dd_tool()` alias retained for notebook compatibility.
- `format_financial_data()`: renders percents xx.xx%, ratios xx.xxx, amounts Rs.##0, lists joined; header notes verify period/currency.
- `valuation_tool(ticker)`:
- Raises if income empty. Uses `columns[0]` as latest period.
- EBITDA fallback Operating Income + Depreciation.
- EV = MarketCap + NetDebt, EV/Revenue, EV/EBITDA.
- Simplified 5-yr projection: 8% EBITDA growth, 10% discount, 3% terminal growth. Explicit LIMITATION: Simplified EBITDA-based academic illustration, not professional FCFF DCF.
- `industry_intelligence_tool()` / `synergy_tool()`: templated qualitative strings for Indian steel; no live market fetch; require evidence for quantification.

5.6 ma_agents/agents.py (52 lines) - LLM layer

- `AgentSuite(settings)`: Groq(api_key), model, max tokens.
- `_complete(role,prompt)`: single chat completion, returns `choices[0].message.content` or "".
- Six methods as per §4.2. No data fetching here - pure writers.

5.7 ma_agents/orchestrator.py (32 lines)

- See §4.1. No retry / parallelism; sequential for clarity and quota control.

5.8 Dependencies (requirements.txt)

- `groq>=0.11,<1.0`, `yfinance>=0.2.40,<2.0`, `pandas>=2.0,<4.0`, `python-dotenv>=1.0,<2.0`, `streamlit>=1.40,<2.0`.
- Python 3.10+ required; Docker uses 3.12-slim.

5.9 Configuration templates

- `.env.example`: `GROQ_API_KEY=your-groq-api-key-here`, `GROQ_MODEL=openai/gpt-oss-20b`, `MAX_COMPLETION_TOKENS=1000`.
- `secrets.toml.example`: TOML quoted form with 700 tokens for Cloud.
- `.dockerignore` / `.gitignore`: exclude `.env`, `.venv/`, `__pycache__`, `reports/*.json` (keep `.gitkeep`).

5.10 Dockerfile (13 lines)

- FROM `python:3.12-slim`, `PYTHONDONTWRITEBYTECODE=1`, `PYTHONUNBUFFERED=1`, `PORT=8501`, `WORKDIR /app`, `pip install`, `COPY . .`, `EXPOSE 8501`, `CMD streamlit run app.py --server.address=0.0.0.0 --server.port=${PORT} --server.headless=true`.
 - Credentials via platform env/secrets, never baked into image.
-

6. Data Sources and Valuation Approach

6.1 Yahoo Finance via yfinance

- Source: `company.info`, `income_stmt`, `balance_sheet`, `cashflow`.
- Currency, period, units taken as returned; header reminds to verify. No primary-source validation.
- Failure modes: missing rows -> NaN -> N/A; empty income -> `ValueError: No income statement data...`; network/auth errors bubble to UI/CLI.

6.2 Ratio logic

- Margins scaled ×100; leverage / coverage as x multiples; guarded division.
- Flags are heuristic triage, not ratings.

6.3 Academic valuation illustration

- Market: MarketCap + NetDebt = EV; EV/Revenue, EV/EBITDA with :.2f x.
 - DCF-style: projects EBITDA, discounts at 10%, Gordon terminal at 3%. Uses EBITDA as cash-flow proxy - retained from notebook to preserve behaviour but labelled non-professional.
 - Required for professional valuation (per prompts): FCFF build, WACC, capex/working-capital, debt schedule, segment forecasts, comps, control premium, sensitivity.
-

7. Setup, Execution and Deployment

7.1 Local setup (PowerShell)

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
Copy-Item .env.example .env
# edit .env -> set GROQ_API_KEY, optional GROQ_MODEL / MAX_COMPLETION_TOKENS
```

Never commit .env or share keys. Rotate any exposed key (notebook embedded key must be treated as exposed).

7.2 Run dashboard

```
streamlit run app.py
# open local URL -> enter acquirer/target/ticker -> Generate analysis (6 model calls,
few minutes)
# Download all reports (JSON)
```

7.3 Run CLI

```
python run.py
python run.py --ticker JINDALSTEL.NS --acquirer "Tata Steel" --target "Jindal Steel"
--output reports/jindal.json
```

7.4 Backend import

```
from dotenv import load_dotenv
from ma_agents import run_analysis
load_dotenv()
reports = run_analysis("JINDALSTEL.NS", "Tata Steel", "Jindal Steel")
```

7.5 Container

Build image, set GROQ_API_KEY as platform secret/env, run on \${PORT}. Add authentication before exposing deal data publicly.

7.6 Streamlit Community Cloud

Live deployment: <https://merger-and-acquisition.streamlit.app/> (app.py entrypoint).

1. Push to GitHub, create app with app.py.
 2. App menu ... -> Settings -> Secrets -> paste from secrets.toml.example (rotated key), Save, Reboot.
 3. If GROQ_API_KEY is required persists: check typos, extra [section], lowercase names - expects top-level GROQ_API_KEY. Keep app private; public use consumes quota.
-

8. Notebook Review Findings Addressed

#	Notebook issue	Resolution in this repo
1	Embedded Groq key in cell	Removed; .env + Secrets + gitignore + rotation warning (README.md:81)
2	financial_dd_tool financial_due_diligence mismatch	vs Both names retained, same implementation (tools.py:84-86)

#	Notebook issue	Resolution in this repo
3	Red-team reran specialists (repeat fetch/calls)	Orchestrator passes reports forward (orchestrator.py:30-31)
4	Inline pip install	requirements.txt
5	EBITDA-discount DCF presented as valuation	Retained but labelled academic illustration, not FCFF DCF (tools.py:136, README.md:85)
6	Missing data / zero denominators / empty statements / token limits	NaN->N/A, guarded ratios, ValueError, _compact_reports, token budget

9. Risks, Ethics, Limitations and Controls

Analytical limits: Qualitative industry/synergy sections are hypothetical; no current market data supplied. Model must not invent figures - prompts enforce, but human verification remains mandatory.

Data risk: Yahoo Finance may lag, restate, or mis-currency; period must be checked per run (Statement period: ... in valuation output).

Model risk: LLM outputs may hallucinate, be inconsistent, or truncate; temperature 0 reduces variance, not error. Red-team + committee stages challenge but do not certify.

Operational: 6 Groq calls per run -> latency, quota, TPM limits. Mitigations: 2400-char chaining cap, 700-1000 token budget, clear rate-limit messaging.

Security / privacy: Dashboard has no auth; deal names are sensitive. Do not bake keys into Docker; use secrets; keep Cloud app private; revoke exposed keys.

Appropriate use: Analyst-assisted academic support only. Requires primary-source diligence before any real decision.

10. Future Work and Roadmap

- Evidence grounding:** Attach statement dates, currency, filing links; cite row values per finding.
- Professional valuation option:** FCFF builder, WACC inputs, comps table, sensitivity tornado.
- Retrieval:** Plug primary sources (annual reports, filings) instead of template industry text.
- Robustness:** Retry/backoff for yfinance/Groq, caching by ticker+period, async specialists, unit tests for _row_value, ratio, _compact_reports, get_settings.
- UX:** Progress per agent, partial-result display, report diff across tickers, PDF export button.
- Governance:** Auth, audit log, PII redaction, quota dashboard, secret rotation runbook.
- Eval:** Golden transaction set, factuality checklist, red-team hit-rate metric.

Appendix A: File Inventory

Path	Lines	Purpose
README.md	93	Overview, setup, Cloud, findings, scope
app.py	125	Streamlit dashboard, error UX, JSON export
run.py	26	CLI entry point
ma_agents/__init__.py	5	Public export run_analysis
ma_agents/orchestrator.py	32	Six-stage pipeline
ma_agents/agents.py	52	Groq writers + context compaction
ma_agents/config.py	56	Env/Secrets settings
ma_agents/tools.py	144	yfinance fetch, ratios, valuation illustration
requirements.txt	5	Pinned ranges
Dockerfile	13	Container for Streamlit
.env.example	5	Local template
secrets.toml.example	5	Cloud template

Path	Lines	Purpose
.dockerignore	7	Exclude secrets/venv/cache/reports JSON
.gitignore	8	Same + ipynb checkpoints, keep .gitkeep
reports/.gitkeep	0	Keeps empty output dir
PROJECT_REPORT.md	this file	Academic report source

Excluded from review: .env (secret), .venv/, __pycache__/, .git/.

Appendix B: Output Schema

run_analysis() returns dict[str,str] (markdown values):

Financial Due Diligence
Valuation
Industry Intelligence
Synergy Analysis
Red-Team Risk Review
Investment Committee Discussion Memo

CLI / dashboard wrap as:

```
{
  "transaction": {"acquirer": "...", "target": "...", "ticker": "...", "generated_at":
"ISO-8601-UTC"},
  "reports": {"Financial Due Diligence": "...", "...": "..."}
}
```

CLI writes reports/analysis.json by default (git-ignored).

Appendix C: References

- Live demo: <https://merger-and-acquisition.streamlit.app/>.
- Project source files listed in Appendix A (workspace snapshot 2026-10-10).
- yfinance, pandas, groq, streamlit, python-dotenv - see requirements.txt ranges.
- Streamlit Community Cloud Secrets docs (referenced in README.md:62-77, app.py:50,71-80).
- Groq Chat Completions API - model default openai/gpt-oss-20b.
- Yahoo Finance - provider of statements/metadata; verify period/currency per run.

End of Report - print this Markdown to PDF for formal submission. Verify provider data and model text against primary sources before reliance.